

SIMATS ENGINEERING

ASSESSMENT TOOL 2 (Medium)- Debugging and Optimization

COURSE NAME: Data Structures

COURSE CODE:CSA03

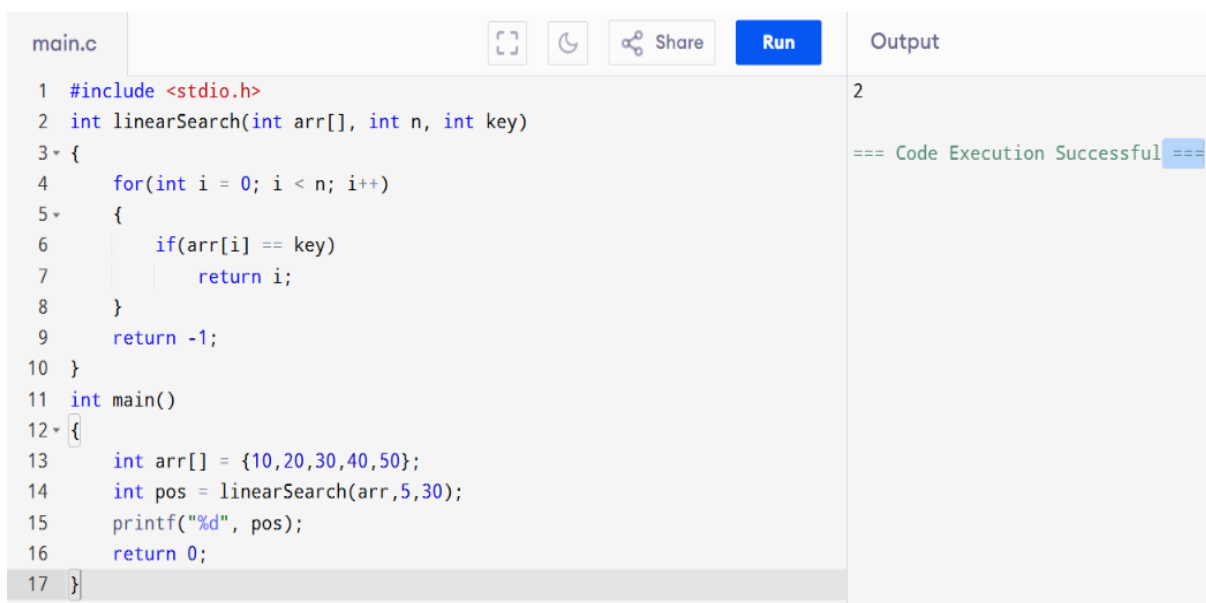
COURSE OUTCOME COVERED

CO2 : Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques. (BTL3, BTL4)

Assessment Tool Weightage: **10%**

1. Find the bugs, include the missing lines in the following snippet.

```
int linearSearch(int arr[], int n, int key) {  
  
for(int i = 0; i <= n; i++) {  
  
if(arr[i] == key)  
  
return i;  
  
}  
  
return -1;  
  
}
```



The screenshot shows a C code editor with a file named 'main.c'. The code defines a linear search function and a main function that tests it. The function has several bugs: it uses ' $\leq$ ' in the loop condition, it has an empty if-statement block, and it returns -1 at the end. The main function calls the function with an array of 5 elements and a key of 30. The output shows '2', indicating the function found the key at index 2. The code execution was successful.

```
main.c  [Icons]  Run  Output  
1 #include <stdio.h>  
2 int linearSearch(int arr[], int n, int key)  
3 {  
4     for(int i = 0; i <= n; i++)  
5     {  
6         if(arr[i] == key)  
7             return i;  
8     }  
9     return -1;  
10 }  
11 int main()  
12 {  
13     int arr[] = {10,20,30,40,50};  
14     int pos = linearSearch(arr,5,30);  
15     printf("%d", pos);  
16     return 0;  
17 }
```

2
=== Code Execution Successful ===

2. Buggy Code in an array

```
#include <stdio.h>

int main() {

    int arr[5] = {10,20,30,40,50};

    for(int i=0;i<=5;i++)

        printf("%d ",arr[i]);

    return 0;

}
```

Expected output:

10 0 30 40
50

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 int main() 4 { 5 int arr[5]={10,20,30,40,50}; 6 7 for(int i=0;i<5;i++) 8 { 9 printf("%d ",arr[i]); 10 } 11 12 return 0; 13 }</pre>		10 20 30 40 50 === Code Execution Successful ===

3. Identify an errors in the following snippet

```
int binarySearch(int arr[], int n, int key)
```

```
{  
    int low = 0, high = n - 1;  
    while(low < high)  
    { int mid = (low + high) / 2;  
      if(arr[mid] == key) return mid;  
      else if(arr[mid] < key)  
          low = mid;  
      else high = mid;  
    }  
    return -1;  
}
```

main.c	Run	Output
<pre>1 #include <stdio.h> 2 int binarySearch(int arr[], int n, int key) 3 { 4 int low=0, high=n-1; 5 while(low<=high) 6 { 7 int mid=(low+high)/2; 8 if(arr[mid]==key) 9 return mid; 10 else if(arr[mid]<key) 11 low=mid+1; 12 else 13 high=mid-1; 14 } 15 16 return -1; 17 } 18 int main() 19 { 20 int arr[]={10,20,30,40,50}; 21 printf("%d",binarySearch(arr,5,40)); 22 return 0; 23 }</pre>	  	3 === Code Execution Successful ===

4. Buggy Code in Stack Push Operation

```
#include <stdio.h>
```

```
#define MAX 5
```

```
int stack[MAX];
```

```
int top = -1;
```

```
void push(int data) {
```

```
    if (top == MAX - 1)
        printf("Stack Overflow");
    else
        stack[top++] = data;
}
```

```
int main() {
    push(10);
    push(20);

    printf("%d", stack[top]);
}
```

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 #define MAX 5 4 5 int stack[MAX]; 6 int top=-1; 7 8 void push(int data) 9 { 10 if(top==MAX-1) 11 printf("Stack Overflow"); 12 else 13 stack[++top]=data; 14 } 15 16 int main() 17 { 18 push(10); 19 push(20); 20 21 printf("%d",stack[top]); 22 23 return 0; 24 }</pre>		20 === Code Execution Successful

5. Buggy Code in Stack POP Operation

```
#include <stdio.h>
#define MAX 5
int stack[MAX];
int top = 2;
int pop() {
    if (top == -1)
        return -1;
```

```
    return stack[++top];  
}
```

main.c	   Share	Run	Output
<pre>1 #include <stdio.h> 2 3 #define MAX 5 4 5 int stack[MAX]={10,20,30}; 6 int top=2; 7 8 int pop() 9 { 10 if(top==0) 11 return -1; 12 13 return stack[top--]; 14 } 15 16 int main() 17 { 18 printf("%d",pop()); 19 20 return 0; 21 }</pre>			30 === Code Execution Successful

6. Buggy Code in underflow operation

```
int pop() {  
    if(top == 0) {  
        printf("Underflow");  
    }
```

```
    return -1;
}
return stack[top--];
}
```

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 int stack[5]; 4 int top=-1; 5 6 int pop() 7 { 8 if(top== -1) 9 { 10 printf("Underflow"); 11 return -1; 12 } 13 14 return stack[top--]; 15 } 16 17 int main() 18 { 19 pop(); 20 21 return 0; 22 }</pre>		Underflow === Code Execution Successful

7. Find the issue in the following code snippet.

```
#include<stdio.h>
```

```
#define MAX 5
```

```
void main()
```

```
{
```



```

int queue[MAX],x;
int front = 0, rear = 0;
printf("Enter the value");
scanf("%d",&x);
void enqueue(int x) {
    if(rear == MAX) {
        printf("Overflow");
        return;
    }
    else{
        queue[rear++] = x;
        printf("Value is %d", queue[rear]);
    }
}
}
}

```

main.c	Run	Output
<pre> 1 #include <stdio.h> 2 #define MAX 5 3 int queue[MAX]; 4 int front=0,rear=-1; 5 void enqueue(int x) 6 { 7 if(rear==MAX-1) 8 { 9 printf("Overflow"); 10 return; 11 } 12 13 queue[++rear]=x; 14 15 printf("Value is %d",queue[rear]); 16 } 17 18 int main() 19 { 20 int x; 21 22 printf("Enter the value: "); 23 scanf("%d",&x); 24 enqueue(x); 25 return 0; 26 } </pre>	Run	<pre> Enter the value: 5 Value is 5 === Code Execution Successful === </pre>

8. Buggy code in node creation

```

#include <stdio.h>
#include <stdlib.h>

```

```

struct Node {
    int data;
    struct Node *next;
};

int main() {
    struct Node *newNode;

    newNode->data = 10;
    newNode->next = NULL;

    printf("%d", newNode->data);

    return 0;
}

```

main.c	Run	Output
<pre> 1 #include <stdio.h> 2 #include <stdlib.h> 3 4 struct Node 5 { 6 int data; 7 struct Node *next; 8 }; 9 10 int main() 11 { 12 struct Node *newNode; 13 14 newNode=(struct Node*)malloc(sizeof(struct Node)); 15 16 newNode->data=10; 17 newNode->next=NULL; 18 19 printf("%d",newNode->data); 20 21 free(newNode); 22 23 return 0; 24 } </pre>		<pre> 10 === Code Execution Success </pre>

9. Include missing statements and display the list of elements in linkedlist

```

void display(struct Node *head)
{
    while(head != NULL);
    {
        printf("%d ", head->data);
        head = head->next;
    }
}

```

```
main.c
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node
5 {
6     int data;
7     struct Node *next;
8 };
9
10 void display(struct Node *head)
11 {
12     while(head != NULL)
13     {
14         printf("%d ", head->data);
15         head = head->next;
16     }
17 }
18
19 int main()
20 {
21     struct Node *head, *second, *third;
22
23     head = (struct Node *)malloc(sizeof(struct Node));
24     second = (struct Node *)malloc(sizeof(struct Node));
25     third = (struct Node *)malloc(sizeof(struct Node));
26
27     head->data = 10;
28     head->next = second;
29
30     second->data = 20;
31     second->next = third;
32
33     third->data = 30;
34     third->next = NULL;
35
36     printf("Linked List Elements: ");
37     display(head);
38
39     free(head);
40     free(second);
41     free(third);
42
43     return 0;
44 }
```

Output

Linked List Elements: 10 20 30

*** Code Execution Successful ***

10. Bug in Queue Full Condition-Identify and fix the errors.

```
#define SIZE 5
int queue[SIZE];
int front = -1, rear = -1;

void enqueue(int item)
{
    if(rear == SIZE-1)
    {
        printf("Queue Full");
        return;
    }

    if(front == -1)
```

```
    front = 0;

    rear = (rear + 1) % SIZE;
    queue[rear] = item;
}
```

main.c		Run	Output
<pre>1 #include <stdio.h> 2 3 #define SIZE 5 4 5 int queue[SIZE]; 6 int front=-1, rear=-1; 7 8 void enqueue(int item) 9 { 10 if((rear+1)%SIZE==front) 11 { 12 printf("Queue Full"); 13 return; 14 } 15 16 if(front==-1) 17 front=0; 18 19 rear=(rear+1)%SIZE; 20 21 queue[rear]=item; 22 } 23 24 int main() 25 { 26 enqueue(10); 27 enqueue(20); 28 29 printf("%d %d", queue[0], queue[1]); 30 31 return 0; 32 }</pre>			<pre>10 20 === Code Execution Successful ===</pre>

Code of Conduct:

I kaviya.G(192524281) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation